ALA:-2

Aim:- Python Deep Dive: Interview Intensive

Python String Concatenation: Technical Deep Dive

1. Strings in CPython

Strings are immutable sequences of Unicode characters.

- **Syntax:**

string_name = "value"

Explanation:

- **Immutable** → cannot be changed after creation
- **Sequence** → ordered collection
- **Unicode** → supports all characters (letters, symbols, emojis)

- **Example:**

```
s = "Hello"
print(s)
```

Every modification creates a new string object.

- **Syntax:**

new_string = old_string + "new_part"

- **Explanation:**
  - Old string remains unchanged
  - New string is created in memory
- **Example:**

```
s = "Hello"
s = s + " World"

print(s)   # Hello World
```

2. Internal Representation (PEP 393)

- **Syntax:**

(No direct syntax — internal behavior)

- **Explanation:**
  - Python chooses smallest memory format automatically
  - **Latin-1 uses 1 byte per character.**
  - Example:
  - `s = "A"`

UCS-2 uses 2 bytes per character.

- **Example:**

s = "Ā"

UCS-4 uses 4 bytes per character (emoji).

- **Example:**

s = " 😊 "

Adding emoji forces entire string upcasting.

- **Syntax:**

string = string + "emoji"

Example:

s = "a" * 1000
s = s + " 😊 "

3. Using + and +=
Concatenation creates a new string and copies data.

- **Syntax:**

result = str1 + str2
str1 += str2

- **Example:**

a = "Hello"
b = "World"

c = a + b
print(c) # HelloWorld

Time complexity becomes O(n²) in loops.

- **Syntax:**

for variable in iterable:
    string += value

- **Example:**

s = ""
for i in range(5):
    s += str(i)
print(s)

+= may optimize if reference count is 1.

- **Syntax:**

(No guaranteed syntax — internal optimization)

- **Explanation:**
  - CPython may reuse memory
  - Not reliable for interviews

4. .join() Method

Used to concatenate multiple strings efficiently.

- **Syntax:**

result = "separator".join(iterable)

Performs two-pass operation (length + copy).

- **Explanation:**
  1. Calculate total length
  2. Allocate memory once
  3. Copy all strings

- **Example:**

```
lst = ["1", "2", "3"]
result = "".join(lst)

print(result)   # 123
```

Time complexity is O(n).

- **Explanation:**
  - Only one allocation
  - No repeated copying

📌 **Example with separator:**

```
words = ["Python", "is", "easy"]
sentence = " ".join(words)

print(sentence)
```

5.f-Strings

Used for string interpolation.

- Syntax:

```
f"string {variable}"
```

Compiled into efficient bytecode (BUILD_STRING).

- **Explanation:**

- Faster execution
- No extra parsing

- **Example:**

```
name = "Devansh"
age = 18

print(f"My name is {name} and I am {age}")
```

Supports expressions inside {}.

- **Example:**

```
a = 5
b = 10

print(f"Sum = {a + b}")
```

6.String Interning

Python reuses identical strings to save memory

- Syntax:

```
a = "text"
b = "text"
```

Comparison becomes O(1) using is.

- **Syntax:**

```
a is b
```

- **Example:**

```
a = "hello"
b = "hello"

print(a is b)   # True
```

Manual interning using sys.intern().

- **Syntax:**

```
import sys
sys.intern(string)
```

- **Example:**

```
import sys

a = sys.intern("hello")
b = sys.intern("hello")

print(a is b)   # True
```

7.StringIO

Provides file-like interface for string building.

- Syntax:

```
from io import StringIO
buffer = StringIO()
```

Use write() to append data.

- **Syntax:**

```
buffer.write(data)
```

Use getvalue() to get final string.

- **Syntax:**

```
result = buffer.getvalue()
```

- **Example:**

```
from io import StringIO

buffer = StringIO()

buffer.write("Hello ")
buffer.write("World")

print(buffer.getvalue())
```

Efficient for large-scale string building.

- **Example:**

```
from io import StringIO

buffer = StringIO()

for i in range(5):
    buffer.write(str(i))

print(buffer.getvalue())   # 01234
```

8. Final Interview One-Liners

- **Strings are immutable → new object on change**
- **+ in loop → O(n²)**
- **.join() → O(n), best for large data**
- **f-strings → fastest for formatting**
- **StringIO → best for heavy operations**
- **Interning → O(1) comparison using is**